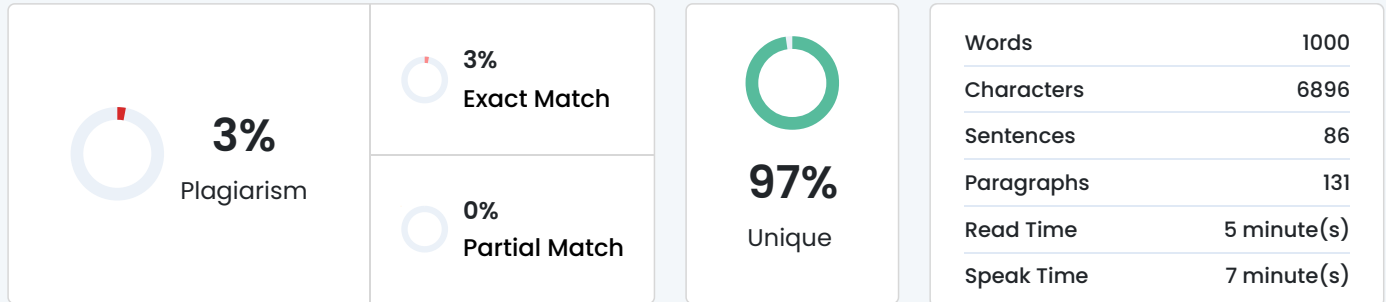


Plagiarism Scan Report



Content Checked For Plagiarism

Smart Hospital System — CSA09 Assignment Report

SAVEETHA UNIVERSITY

Department of Computer Science and Engineering

COMMON COURSE ASSIGNMENT REPORT

Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java

Course Code & Name: CSA09 — Programming in Java (Slot B)

Submitted by (Team):

1. _____Anish Ebenezer_____ (Reg. No: 192511337) — Module 1 Lead
2. _____Pushparaj B_____ (Reg. No: 192421136) — Module 2 Lead
3. _____N.NESARAJAN_____ (Reg. No: 192511386) — Module 3 Lead
4. _____Asif ali khan_____ (Reg. No: 192521009) — Module 4 Lead
5. _____G Sreekanth_____ (Reg. No:192311422) — Module 5 Lead

Faculty: Dr. Thalpathi Rajasekaran R

A. Assignment Information

Department

Computer Science and Engineering

Programme

Computer Science and Engineering

Course Code & Course Name

CSA09 — Programming in Java (Slot B)

Academic Year / Batch

2025-2029

Faculty Name

Dr. Thalpathi Rajasekaran

Assignment Title

Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java

Maximum Marks

100

Course Outcome(s) — CO

CO1: OOP principles (classes, encapsulation, inheritance, polymorphism). CO2: Collection Framework (List, ArrayList, Set, Map, HashMap, Hashtable) with Iterator/ListIterator and generics. CO3: Multithreading (priorities, synchronization, inter-thread communication) and exception handling.

Bloom's Taxonomy Level

L2 — Understand; L3 — Apply; L4 — Analyse

SDG Mapping (SDG 1–17)

SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure; SDG 10 – Reduced Inequalities

Industry / Societal Relevance

Digitises hospital front-office and pharmacy workflows that are still largely paper-based in many small and mid-sized clinics, reducing appointment conflicts, medicine stock-outs and missed follow-ups.

Table of Contents

A. Assignment Information	1
Table of Contents	1
1. Problem Statement and Problem Formulation	1
1.1 Problem Statement	1
1.2 Problem Formulation	1
2. Objective and Expected Outcomes	1
2.1 Objectives	1
2.2 Expected Outcomes	1
3. Requirements, Constraints and Assumptions	1
3.1 Functional Requirements	1
3.2 Non-Functional / Technical Constraints	1
3.3 Assumptions	1
4. Application of Relevant Course Knowledge / Concepts	1
4.1 CO1 – OOP: Classes, Encapsulation, Inheritance, Polymorphism	1
4.2 CO2 – Collection Framework, Generics, Iterators	1
4.3 CO3 – Exception Handling, Multithreading, Synchronization	1
5. Design / Proposed Solution / Methodology	1
5.1 Architectural Overview	1
5.2 Class Design Summary	1
5.3 Key Design Decisions and Alternatives Considered	1
6. Algorithm / Pseudocode / Flowchart	1
6.1 Flowchart – Book Appointment (described)	1
7. Implementation / Source Code and Environment / Tools Used	1
7.1 Development Environment	1
7.2 Representative Source Code	1
7.2.1 Module 1 – Polymorphic factory (Medicine)	1
7.2.2 Module 2 – Collections-backed booking (HospitalService)	1
7.2.3 Module 3 – Exception-safe menu dispatch (ConsoleApp)	1
7.2.4 Module 4 – Thread-safe stock reduction (Inventory + JDBC)	1
7.2.5 Module 5 – Menu loop and Applet action dispatch	1
8. Test Cases and Expected/Actual Results	1
9. Execution Screenshots / Outputs	1
10. Results and Validation	1
10.1 Functional Coverage	1
10.2 Observations	1
11. Analysis, Comparison, Trade-offs and Justification of the Final Solution	1
11.1 Design Alternatives Considered	1
11.2 Trade-offs of the Final Solution	1
11.3 Justification	1
12. Broader Considerations / SDG Relevance	1
12.1 SDG 3 – Good Health and Well-being	1
12.2 SDG 9 – Industry, Innovation and Infrastructure	1
12.3 SDG 10 – Reduced Inequalities	1
12.4 Ethics, Safety and Professional Responsibility	1
13. Conclusion, Limitations and Possible Improvements	1
13.1 Conclusion	1

13.2 Limitations	1
13.3 Possible Improvements	1
14. Individual Contribution of Group Members	1
Member 1 – Core entity classes and inheritance hierarchy	1
Member 2 – The central service facade and its collections	1
Member 3 – Custom exceptions and the notification class hierarchy	1
Member 4 – Concurrency primitives and the database access layer	1
Member 5 – Console and Applet front ends, entry point, overall test coordination	1
15. References	1
16. One-Page Individual Reflection	1
16.1 Individual Reflection – Member 1	1
Design/Development Decisions	1
Challenges Faced	1
Learning Gained	1
SDG Connection	1
Course Outcome Attainment	1
16.2 Individual Reflection – Member 2	1
Design/Development Decisions	1
Challenges Faced	1
Learning Gained	1
SDG Connection	1
Course Outcome Attainment	1
16.3 Individual Reflection – Member 3	1
Design/Development Decisions	1
Challenges Faced	1
Learning Gained	1
SDG Connection	1
Course Outcome Attainment	1
16.4 Individual Reflection – Member 4	1
Design/Development Decisions	1
Challenges Faced	1
Learning Gained	1
SDG Connection	1
Course Outcome Attainment	1
16.5 Individual Reflection – Member 5	1
Design/Development Decisions	1
Challenges Faced	1
Learning Gained	1
SDG Connection	1
Course Outcome Attainment	1

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Hospitals and clinics that rely on manual registers for patient appointments and pharmacy stock face recurring, avoidable problems: doctors get double-booked or under-booked because free slots are not visible at a glance; patients with genuine follow-up needs are turned away when a doctor is full instead of being queued; medicine stock is dispensed without a reliable check against what is actually on the shelf, leading to stock-outs discovered only when a patient is standing at the counter; and nobody is reminded of an upcoming appointment or a falling stock level until it is too late. The assignment requires the design, implementation and evaluation of a menu-driven Java application – the Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System – that automates this workflow end-to-end: registering patients and doctors, adding and tracking medicines, displaying available slots and stock, booking and cancelling appointments with a waitlist fallback, dispensing medicine with stock validation,

generating patient-visit and pharmacy-utilization reports, and dispatching appointment-reminder and low-stock notifications, all backed by persistent JDBC storage.

1.2 Problem Formulation

- * Core entities to model: Patient (with sub-categories), Doctor, Appointment, Medicine (with sub-types), Inventory and Notification.
- * Core workflow to automate: register → discover availability → book/cancel → dispense → report → notify, each step guarded by validation and exception handling.
- * Concurrency to simulate: two patients competing for the same doctor slot, and a background dispatcher delivering queued notifications, without corrupting shared appointment or stock data.
- * Persistence to provide: every create/update operation must survive a restart, via JDBC against an

Matched Source

Similarity 3%

Title: [Texas.gov](#)

Sep 2, 2025 — All locally developed courses must be assigned an eight-digit course code. The number may be a combination of letters and numbers, the last ...

[Check](#)

Check By:  Dupli Checker